Google Antigravity y la Automatización de Fichas de Productos y Procesos

Consulta. Bajos los alcances de la NT de Implementación de la Gestión por Procesos debo de elaborar fichas de Productos y Procesos, en tal sentido, Google Antigravity puede manipular archivos en la nube o archivos en carpetas locales

Hola. Es excelente que estés trabajando en la implementación de la Gestión por Procesos (probablemente bajo los alcances de la Norma Técnica de la Secretaría de Gestión Pública - PCM en Perú). Entiendo que la elaboración de las fichas de Productos y Procesos requiere bastante organización documental.

Sin embargo, debo hacer una aclaración importante para que no pierdas tiempo buscando la herramienta equivocada: **"Google Antigravity" no es un programa, ni un servicio en la nube, ni una herramienta de gestión de archivos.** ### ¿Qué es realmente Google Antigravity?

Es simplemente un *Easter egg* (una broma o truco de internet) creado hace muchos años por un desarrollador web llamado Mr. Doob. Si lo buscas y haces clic en él, verás que todos los elementos de la página del buscador de Google "caen" hacia el fondo de la pantalla como si fueran afectados por la gravedad. **No tiene ninguna utilidad práctica ni capacidad para manipular archivos.**

Herramientas reales de Google para tu proyecto

Si necesitas manipular archivos en la nube o sincronizarlos con tus carpetas locales para elaborar tus Fichas de Procesos, el ecosistema correcto es **Google Workspace**. Aquí te detallo lo que realmente necesitas:

- **Google Drive:** Es el servicio de almacenamiento en la nube donde puedes guardar, organizar y compartir todos los documentos relacionados con tu Norma Técnica.
- **Google Drive para Escritorio (Drive for Desktop):** Esta es la aplicación que debes instalar en tu computadora. Te permite sincronizar tus carpetas locales (en Windows o Mac) directamente con la nube. Cualquier cambio que hagas en un archivo local se actualizará automáticamente en Drive.
- **Google Docs y Google Sheets (Documentos y Hojas de cálculo):** Son las herramientas ideales para diseñar y llenar las fichas de caracterización de procesos, flujogramas y matrices de productos. Varios miembros de tu equipo pueden editarlas al mismo tiempo.

Como inteligencia artificial, yo (Gemini) también puedo ayudarte muchísimo en este proceso. Aunque no puedo entrar a tu computadora a mover archivos locales de una carpeta a otra, puedo ayudarte a redactar, analizar y estructurar la información técnica que va dentro de esas fichas.

¿Te gustaría que te ayude a estructurar una plantilla base para tus Fichas de Procesos y Productos según los lineamientos de la Norma Técnica, o prefieres que te explique cómo configurar Google Drive en tu equipo para organizar estos documentos?

Consulta. Me gustaría elaborar estás fichas en formato JSON y que luego se puedan de alguna forma cargarse a la plantillas e ir confirmando los documentos de procesos

Trabajar las Fichas de Procesos y Productos (Anexo 2 de la NT N° 002-2025-PCM/SGP) utilizando JSON como base de datos y plantillas dinámicas es una excelente estrategia arquitectónica. Este enfoque de automatización encaja perfectamente con metodologías de desarrollo ágiles, permitiéndote escalar la documentación sin tareas manuales repetitivas.

Aquí te presento cómo estructurar este modelo y las vías técnicas para inyectar los datos en tus plantillas.

1. Estructura JSON para la Ficha de Proceso

Podemos modelar el JSON basándonos en los campos requeridos por la Secretaría de Gestión Pública y los datos de la institución. A continuación, un ejemplo estructurado utilizando los datos del proceso misional M.01.01 "Capacitación para el fortalecimiento de capacidades asociativas y empresariales" y su producto oficial asociado:

JSON

```
{
  "fichaProceso": {
    "codigo": "M.01.01",
    "denominacion": "Capacitación para el fortalecimiento de capacidades asociativas y empresariales",
    "tipoProceso": "Misional",
    "duenoProceso": "Unidad de Promoción y Desarrollo de Capacidades",
    "objetivo": "Desarrollar y fortalecer las competencias operativas y comerciales de los productores agrarios.",
    "alcance": "Comprende desde la planificación de la demanda de capacitación hasta la evaluación de satisfacción de los participantes.",
    "entradas": [
      {
        "proveedor": "Organizaciones Agrarias",
        "insumo": "Requerimientos de fortalecimiento de capacidades"
      }
    ],
    "salidas": [
      {
        "cliente": "Organizaciones Agrarias",
        "productoOficial": "Capacitación para fortalecimiento de capacidades empresariales productivas, asociativas y de posicionamiento al mercado"
      }
    ],
    "recursos": {
      "humanos": ["Especialistas en promoción", "Capacitadores externos"],
```

```
    "tecnologicos": ["Sistema ESV-SIPA", "Plataformas de videoconferencia"]
  },
  "indicadores": [
    {
      "nombre": "Porcentaje de ejecución del plan de capacitaciones",
      "formula": "(Capacitaciones ejecutadas / Capacitaciones programadas) * 100"
    }
  ]
 }
}
```

2. ¿Cómo cargar este JSON a las Plantillas?

Para lograr que estos datos autocompleten los documentos y puedas ir confirmándolos visualmente, tienes dos rutas principales dependiendo de dónde prefieras que se ejecute la solución:

Ruta A: En la Nube (Google Workspace + Apps Script)

Esta es la ruta más directa y colaborativa.

- **La Plantilla:** Creas un Google Doc que servirá como tu formato vacío de la Ficha. En lugar de texto fijo, colocas variables o "tags" como `{{codigo}}`, `{{denominacion}}`, `{{objetivo}}`.
- **El Motor (Apps Script):** Escribes un pequeño script asociado a un Google Sheet o directamente en Drive. El script lee tu archivo JSON, crea una copia de la plantilla original y utiliza la función nativa `body.replaceText('{{codigo}}', json.fichaProceso.codigo)` para inyectar los datos exactos.
- **El Resultado:** Se genera automáticamente un Google Doc nuevo por cada proceso declarado en tu JSON, listo para ser revisado por los dueños de proceso y exportado a PDF.

Ruta B: En Entorno Local o Servidor (PHP / Vanilla JS)

Si prefieres integrar esto en un aplicativo o módulo web de uso interno:

- **Con Vanilla JS / Node.js:** Puedes usar librerías como `docxtemplater`. Creas una plantilla en formato `.docx` (Word) con etiquetas como `{#codigo}`, subes el JSON desde tu interfaz, y la librería procesa y genera el documento `.docx` ya rellenado para su descarga.
- **Con PHP:** Utilizando tu entorno habitual (PHP con PDO y MySQL), puedes almacenar la caracterización de los procesos en base de datos, generar el JSON al vuelo cuando se requiera, y usar una librería como `PHPWord` (o `phpdocx`) para mapear esos datos hacia una plantilla `.docx` alojada en el servidor.

Esta lógica de separar los **datos** (JSON) de la **vista** (Plantilla Docx/Google Doc) te permitirá actualizar masivamente el diseño estructural de las fichas en el futuro sin tener que reescribir manualmente la información técnica de los procesos.

¿Te gustaría que generemos el código base en **Google Apps Script** para automatizar tus plantillas directamente en Drive, o prefieres abordar la lógica para un entorno local con **Vanilla JS o PHP**?

Consulta. Prefiero la opción 1 usando una apps Script para una plantilla cargada en Google Drive, generar apps Script para esta opción

Esta solución automatizada mediante **Google Apps Script** te permitirá actuar como un orquestador de datos, manteniendo la integridad de la información en el JSON y la presentación en el Google Doc.

Para que este script funcione, primero debes crear un **Google Doc** que servirá de plantilla, utilizando etiquetas entre llaves dobles, por ejemplo: `{{codigo}}`, `{{denominacion}}`, `{{duenoProceso}}`, etc.

Google Apps Script: Generador de Fichas de Procesos

Puedes pegar este código en el editor de scripts de una Hoja de Cálculo de Google o como un script independiente en script.google.com.

JavaScript

```javascript
/**
 * Automatización de Fichas de Procesos para Modernización Institucional
 * Genera documentos a partir de un objeto JSON y una plantilla de Google Docs.
 */

function generarFichasDesdeJSON() {
  // 1. Configuración de IDs (Reemplaza con tus IDs de Google Drive)
  const TEMPLATE_ID = 'ID_DE_TU_PLANTILLA_DOC';
  const DESTINATION_FOLDER_ID = 'ID_DE_LA_CARPETA_DE_DESTINO';

  // 2. Datos de ejemplo (Este JSON puede ser leído desde un archivo .json en Drive)
  const data = [
    {
      "codigo": "M.01.01",
      "denominacion": "Capacitación para el fortalecimiento de capacidades asociativas",
      "tipoProceso": "Misional",
      "duenoProceso": "Unidad de Promoción y Desarrollo de Capacidades",
      "objetivo": "Desarrollar competencias operativas en los productores.",
      "fecha": "18/04/2026"
    },
    {
      "codigo": "S.02.04",
      "denominacion": "Gestión de la Innovación Pública - Proyecto V-RAEJI",
      "tipoProceso": "Soporte / Innovación",
      "duenoProceso": "Unidad de Planeamiento y Presupuesto",
      "objetivo": "Virtualización y evaluación de riesgos en procesos de gestión pública.",
      "fecha": "18/04/2026"
    }
```

```
  ];

  const templateFile = DriveApp.getFileById(TEMPLATE_ID);
  const destinationFolder = DriveApp.getFolderById(DESTINATION_FOLDER_ID);

  data.forEach(item => {
    // Crear una copia de la plantilla para cada proceso
    const nombreArchivo = `Ficha de Proceso - ${item.codigo} - ${item.denominacion}`;
    const copy = templateFile.makeCopy(nombreArchivo, destinationFolder);
    const doc = DocumentApp.openById(copy.getId());
    const body = doc.getBody();

    // Reemplazar los tags en el documento con los datos del JSON
    // El método utiliza encadenamiento para mayor limpieza
    body.replaceText('{{codigo}}', item.codigo);
    body.replaceText('{{denominacion}}', item.denominacion);
    body.replaceText('{{tipoProceso}}', item.tipoProceso);
    body.replaceText('{{duenoProceso}}', item.duenoProceso);
    body.replaceText('{{objetivo}}', item.objetivo);
    body.replaceText('{{fecha}}', item.fecha);

    // Guardar y cerrar para confirmar cambios
    doc.saveAndClose();
    console.log(`Documento generado: ${nombreArchivo}`);
  });
}

/**
 * Función opcional para leer un archivo JSON directamente desde una carpeta de Drive
 */
function cargarJSONDesdeDrive(fileId) {
  const file = DriveApp.getFileById(fileId);
  const jsonContent = file.getBlob().getDataAsString();
  return JSON.parse(jsonContent);
}
```

Instrucciones para la Implementación

1. **Preparación de la Plantilla:** Crea un documento con el diseño oficial de la NT de Implementación de Gestión por Procesos. En los espacios donde debe ir la información técnica, escribe los tags exactamente como aparecen en el script (ej: `{{duenoProceso}}`). Es vital que los nombres de las unidades orgánicas coincidan con el Manual de Operaciones (MOP) vigente.
2. **Obtención de IDs:** * El **ID del documento** está en la URL: `docs.google.com/document/d/XXXXXXXX/edit`.
    - El **ID de la carpeta** está en la URL de Drive: `drive.google.com/drive/folders/YYYYYYYY`.
3. **Ejecución:** Al ejecutar la función `generarFichasDesdeJSON` por primera vez, Google te pedirá permisos para acceder a Drive y Documentos. Una vez

concedidos, el script procesará el array y depositará los nuevos archivos en la carpeta de destino.

Ventajas de este flujo

- **Confirmación Documental:** Una vez generados, puedes abrir los documentos para realizar ajustes finos o validaciones técnicas antes de la firma oficial.
- **Consistencia de Datos:** Si el código de un proceso cambia en el inventario central (el JSON), simplemente vuelves a correr el script y generas versiones actualizadas sin errores de transcripción.
- **Escalabilidad:** Puedes añadir campos complejos al JSON, como arreglos de "Entradas" o "Salidas", y programar el script para que inserte filas en tablas de forma dinámica.

¿Te gustaría profundizar en cómo manejar tablas dinámicas en el script para las secciones de "Entradas" y "Salidas" de la ficha?

Consulta. Y se podrá implementar este flujo de trabajo como una aplicación app mi idea es generar tantos archivos JSON como procesos tengan mi inventario de procesos y procedimientos que es una tabla Excel que podría pasar a un formato JSON

Es totalmente factible y, de hecho, es la evolución natural para un **Especialista en Modernización**. Al transformar tu inventario de Excel en una serie de objetos JSON, estás creando un "decoupled architecture" donde los datos son independientes del formato final (el documento).

Aquí tienes la hoja de ruta técnica para convertir esta idea en una aplicación funcional dentro del ecosistema que ya manejas:

1. El Flujo de Datos: De Excel a la Ficha Final

Para implementar esto como una aplicación (Web App), el flujo ideal sería:

1. **Carga:** Importas tu Excel de inventario a una **Google Sheet**.
2. **Procesamiento:** Un script recorre cada fila (un proceso/procedimiento).
3. **Serialización:** Convierte cada fila en un objeto JSON.
4. **Almacenamiento/Generación:** Guarda el JSON en una carpeta de Drive y, opcionalmente, dispara la creación del Google Doc.

2. Conversión de Inventario (Excel) a JSON

Si tienes tu inventario en una tabla, puedes usar este fragmento de código para "mapear" las columnas de Excel a claves JSON. Esto es puro **VibeCoding**: rápido, funcional y estructurado.

JavaScript

```
function transformarInventarioAJSON() {
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Inventario");
  const data = sheet.getDataRange().getValues();
```

```
  const headers = data[0]; // Asume que la primera fila son los encabezados

  const carpetaJson = DriveApp.getFolderById("ID_CARPETA_JSONS");

  for (let i = 1; i < data.length; i++) {
    let objetoProceso = {};
    data[i].forEach((valor, index) => {
      // Limpiamos los nombres de encabezados para que sean claves JSON válidas
      let clave = headers[index].toString().toLowerCase().replace(/ /g, "_");
      objetoProceso[clave] = valor;
    });

    // Crear el archivo JSON individual en Drive
    const nombreArchivo = `proceso_${objetoProceso.codigo || i}.json`;
    carpetaJson.createFile(nombreArchivo, JSON.stringify(objetoProceso, null, 2),
MimeType.PLAIN_TEXT);

    console.log(`JSON generado para: ${nombreArchivo}`);
  }
}
```

3. Implementación como "Web App" (Interfaz de Usuario)

Para que esto se sienta como una aplicación real y no solo un script oculto, puedes publicar el script como una **Web App**. Esto te permite tener un panel de control (Dashboard) hecho con **HTML y Vanilla JavaScript**.

Estructura del Proyecto en Apps Script:

- `Código.gs`: Lógica del servidor (leer Drive, generar archivos).
- `Index.html`: Interfaz con botones para "Sincronizar Inventario" o "Generar Fichas".

Ejemplo de la Interfaz (Index.html):

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <base target="_top">
    <style>
      body { font-family: 'Segoe UI', sans-serif; padding: 20px; background-color: #f4f7f6; }
      .card { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 2px 10px
rgba(0,0,0,0.1); }
      button { background: #2ecc71; color: white; border: none; padding: 10px 20px; border-radius:
4px; cursor: pointer; }
      button:hover { background: #27ae60; }
    </style>
  </head>
  <body>
```

```
  <div class="card">
    <h2>Panel de Gestión por Procesos - AGROIDEAS</h2>
    <p>Utilice este panel para procesar el inventario y confirmar documentos.</p>
    <button onclick="ejecutarProceso()">Generar JSONs y Fichas</button>
    <div id="status"></div>
  </div>

  <script>
    function ejecutarProceso() {
      document.getElementById('status').innerText = "Procesando...";
      google.script.run
          .withSuccessHandler(msg => document.getElementById('status').innerText = "¡Éxito! " +
msg)
        .transformarInventarioAJSON(); // Llama a la función en Código.gs
    }
  </script>
 </body>
</html>
```

4. ¿Por qué este enfoque es superior para la Modernización?

1. **Interoperabilidad:** Esos archivos JSON que generes pueden ser consumidos luego por otros módulos de tu proyecto **ESV-SIPA** o incluso por el motor de simulación **V-RAEJI**.
2. **Trazabilidad:** Al tener un JSON por proceso, puedes implementar un control de versiones simple. Si la norma técnica de la PCM cambia un campo, solo actualizas la lógica de mapeo y vuelves a generar todo en segundos.
3. **Escalabilidad:** Puedes pasar de manejar 10 procesos a 500 sin incrementar el esfuerzo manual.

Este flujo no solo cumple con la **NT de Implementación de Gestión por Procesos**, sino que eleva el estándar de la oficina hacia una gestión pública basada en datos y no solo en documentos aislados.

¿Te gustaría que diseñemos el mapeo específico de columnas basándonos en los campos reales de tu Excel de inventario?

Consulta. Si claro tengo el inventario ya en unas carpetas en la nube de Google Drive

¡Excelente! Al tener ya los inventarios organizados en carpetas de Google Drive, la integración para crear la aplicación web es directa y nativa, eliminando la necesidad de subir archivos manualmente.

Para materializar esta solución y generar los archivos JSON de manera automatizada, construiremos la aplicación dividida en dos partes dentro de Google Apps Script: el **Backend** (la lógica que procesa los datos en los servidores de Google) y el **Frontend** (la interfaz visual que usarás tú o tu equipo).

Aquí tienes la arquitectura del código lista para implementarse y procesar la matriz del inventario:

1. El Motor de Datos (Código.gs - Backend)

Este script se encargará de conectarse a la carpeta específica de Drive, leer tu tabla de inventario y generar un archivo JSON individual por cada proceso, estructurando correctamente los datos como Código, Denominación (Niveles 1, 2 y 3) y el Producto Oficial asociado.

JavaScript

```
// Configuración de IDs (Reemplazar con los IDs reales de tus carpetas y archivos en Drive)
const ID_CARPETA_INVENTARIOS = 'PONER_AQUI_ID_CARPETA_ORIGEN';
const ID_CARPETA_JSON_DESTINO = 'PONER_AQUI_ID_CARPETA_DESTINO';
const NOMBRE_ARCHIVO_INVENTARIO = 'Inventario_Procesos'; // Nombre parcial o total del archivo

/**
 * Función principal que sirve la interfaz web (Frontend)
 */
function doGet() {
  return HtmlService.createHtmlOutputFromFile('Index')
      .setTitle('Generador JSON - Gestión por Procesos')
      .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}

/**
 * Función llamada desde la interfaz para procesar el inventario
 */
function procesarInventarioHaciaJSON() {
  try {
    const carpetaOrigen = DriveApp.getFolderById(ID_CARPETA_INVENTARIOS);
    const archivos = carpetaOrigen.searchFiles(`title contains '${NOMBRE_ARCHIVO_INVENTARIO}'`);

    if (!archivos.hasNext()) {
      return "Error: No se encontró el archivo de inventario en la carpeta especificada.";
    }

    const archivoExcel = archivos.next();
    // Asumimos que el archivo es un Google Sheet (si es Excel, debe abrirse como Sheet)
    const spreadsheet = SpreadsheetApp.openById(archivoExcel.getId());
    const sheet = spreadsheet.getSheets()[0]; // Lee la primera pestaña
    const data = sheet.getDataRange().getValues();

    // Obtener encabezados y limpiar espacios
    const headers = data[0].map(h => h.toString().toLowerCase().trim().replace(/\s+/g, "_"));
    const carpetaDestino = DriveApp.getFolderById(ID_CARPETA_JSON_DESTINO);

    let contador = 0;

    // Iterar sobre cada fila del inventario (saltando los encabezados)
```

```
  for (let i = 1; i < data.length; i++) {
    let fila = data[i];
    let objetoProceso = {};

    // Mapeo dinámico basado en las columnas
    headers.forEach((header, index) => {
      if(fila[index]) {
        objetoProceso[header] = fila[index];
      }
    });

    // Solo generar JSON si la fila tiene un código válido (evitar filas vacías)
                    if (objetoProceso.codigo_nivel_1 || objetoProceso.codigo_nivel_2 ||
objetoProceso.codigo_nivel_3) {
      // Usar el código más profundo disponible para el nombre del archivo
            let codigoFinal = objetoProceso.codigo_nivel_3 || objetoProceso.codigo_nivel_2 ||
objetoProceso.codigo_nivel_1 || `proc_${i}`;
      let nombreArchivo = `${codigoFinal}.json`;

              carpetaDestino.createFile(nombreArchivo, JSON.stringify(objetoProceso, null, 2),
MimeType.PLAIN_TEXT);
      contador++;
    }
  }

    return `Operación exitosa. Se han generado ${contador} archivos JSON de los procesos
inventariados.`;

 } catch (error) {
   return `Error en el procesamiento: ${error.message}`;
 }
}
```

2. La Interfaz de Usuario (Index.html - Frontend)

Esta es la vista de la aplicación. Es un panel sencillo y profesional construido con HTML y CSS puro, alineado a metodologías ágiles, que permite ejecutar el motor de datos con un clic.

HTML

```
<!DOCTYPE html>
<html>
 <head>
   <base target="_top">
   <meta charset="UTF-8">
   <title>Generador de Fichas</title>
   <style>
     :root {
       --primary-color: #0f9d58; /* Verde Google */
```

```
    --hover-color: #0b8043;
    --bg-color: #f8f9fa;
    --text-color: #202124;
  }
  body {
    font-family: 'Google Sans', Roboto, Arial, sans-serif;
    background-color: var(--bg-color);
    color: var(--text-color);
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
  }
  .dashboard-card {
    background: white;
    padding: 40px;
    border-radius: 12px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    text-align: center;
    max-width: 500px;
    width: 100%;
  }
  h2 { margin-top: 0; color: #1a73e8; }
  p { font-size: 14px; color: #5f6368; line-height: 1.5; margin-bottom: 24px; }
  .btn-procesar {
    background-color: var(--primary-color);
    color: white;
    border: none;
    padding: 12px 24px;
    font-size: 16px;
    font-weight: bold;
    border-radius: 6px;
    cursor: pointer;
    transition: background-color 0.3s;
    width: 100%;
  }
  .btn-procesar:hover { background-color: var(--hover-color); }
  .btn-procesar:disabled { background-color: #dadce0; cursor: not-allowed; }
  #mensaje-estado {
    margin-top: 20px;
    padding: 12px;
    border-radius: 6px;
    font-size: 14px;
    display: none;
  }
  .exito { background-color: #e6f4ea; color: #137333; border: 1px solid #ceead6; }
  .error { background-color: #fce8e6; color: #c5221f; border: 1px solid #fad2cf; }
  .cargando { background-color: #e8f0fe; color: #1967d2; border: 1px solid #d2e3fc; }
</style>
```

```
 </head>
 <body>
  <div class="dashboard-card">
    <h2>Automatización de Fichas de Procesos</h2>
      <p>Este módulo lee el inventario estructurado desde Google Drive y serializa cada fila en objetos JSON individuales para la futura inyección en plantillas dinámicas.</p>

    <button id="btnProcesar" class="btn-procesar" onclick="iniciarProcesamiento()">
      Generar JSON desde Inventario
    </button>

    <div id="mensaje-estado"></div>
  </div>

  <script>
    function iniciarProcesamiento() {
      const btn = document.getElementById('btnProcesar');
      const estado = document.getElementById('mensaje-estado');

      // Estado de carga
      btn.disabled = true;
      btn.innerText = "Procesando matriz...";
      estado.style.display = "block";
      estado.className = "cargando";
      estado.innerText = "Leyendo carpetas en Drive y estructurando datos...";

      // Llamada al backend
      google.script.run
        .withSuccessHandler(function(respuesta) {
          btn.disabled = false;
          btn.innerText = "Generar JSON desde Inventario";
          estado.className = respuesta.includes("Error") ? "error" : "exito";
          estado.innerText = respuesta;
        })
        .withFailureHandler(function(error) {
          btn.disabled = false;
          btn.innerText = "Generar JSON desde Inventario";
          estado.className = "error";
          estado.innerText = "Fallo crítico en el servidor: " + error.message;
        })
        .procesarInventarioHaciaJSON();
    }
  </script>
 </body>
</html>
```

¿Cómo desplegar esta App?

1. Abre un nuevo proyecto en script.google.com.

2. Pega el código backend en el archivo Código.gs.
3. Crea un nuevo archivo HTML llamado Index.html y pega el código del frontend.
4. Reemplaza los IDs de las carpetas (ID_CARPETA_INVENTARIOS y ID_CARPETA_JSON_DESTINO) en Código.gs con los identificadores reales de tus carpetas de Drive (los encuentras en la URL de la carpeta: drive.google.com/drive/folders/ESTE_ES_EL_ID).
5. Ve a la esquina superior derecha, haz clic en **Implementar > Nueva implementación**.
6. Selecciona el tipo **Aplicación web**. Configura el acceso para que lo ejecutes tú o cualquier usuario con el enlace dentro de la institución.
7. Al implementarlo, te dará una URL. Esa URL es tu nueva aplicación funcionando en la nube.

Este modelo asegura que cada vez que actualices el inventario general, con un solo clic tendrás todos los objetos JSON actualizados y listos para alimentar los sistemas o generar las Fichas Documentales.

Consulta. Con toda está información proyecta el aplicativo

Voy a proyectar la aplicación completa. Para que puedas visualizar exactamente cómo quedará y cómo funcionará, he construido un **Prototipo Interactivo (Dashboard)**.

Este prototipo te permite simular el proceso de lectura de tu Excel y la generación de los JSON, tal como lo hará la aplicación real en Google Drive. Además, en la pestaña **"Código de Implementación"** dentro de la misma aplicación, encontrarás el código exacto que deberás copiar y pegar en tu Google Apps Script para que funcione en la vida real.

Aquí tienes el aplicativo proyectado:

```
import React, { useState, useEffect } from 'react';
import {
  FolderDown,
  FileJson,
  Play,
  CheckCircle,
  Settings,
  Database,
  FileText,
  Code,
  Copy,
  AlertCircle
} from 'lucide-react';

const App = () => {
  const [activeTab, setActiveTab] = useState('dashboard');
  const [isProcessing, setIsProcessing] = useState(false);
  const [procesosGenerados, setProcesosGenerados] = useState([]);
```

```
  const [jsonSeleccionado, setJsonSeleccionado] = useState(null);
  const [copiado, setCopiado] = useState('');
  const [fileName, setFileName] = useState('');

  const handleFileUpload = (event) => {
    const file = event.target.files[0];
    if (!file) return;

    setFileName(file.name);
    setIsProcessing(true);

    const reader = new FileReader();
    reader.onload = (e) => {
      try {
        const content = e.target.result;
        const parsedData = JSON.parse(content);
         // Garantizar que siempre manejemos un array, incluso si suben
un solo objeto
           const dataArray = Array.isArray(parsedData) ? parsedData :
[parsedData];

        setTimeout(() => {
          setProcesosGenerados(dataArray);
          setJsonSeleccionado(null);
          setIsProcessing(false);
        }, 800); // Pausa breve para feedback visual de carga
      } catch (error) {
             alert("Error al leer el archivo JSON. Verifica que la
estructura tenga un formato válido.");
        setIsProcessing(false);
      }
    };
    reader.readAsText(file);
  };

  const copiarCodigo = (id, texto) => {
    navigator.clipboard.writeText(texto);
    setCopiado(id);
    setTimeout(() => setCopiado(''), 2000);
  };

  const codigoBackend = `// 1. EL MOTOR DE DATOS (Código.gs - Backend)
const ID_CARPETA_INVENTARIOS = 'PONER_AQUI_ID_CARPETA_ORIGEN';
```

```
const ID_CARPETA_JSON_DESTINO = 'PONER_AQUI_ID_CARPETA_DESTINO';
const NOMBRE_ARCHIVO_INVENTARIO = 'Inventario_Procesos';

function doGet() {
  return HtmlService.createHtmlOutputFromFile('Index')
      .setTitle('Generador JSON - NT Gestión por Procesos')
      .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}

function procesarInventarioHaciaJSON() {
  try {
                                    const         carpetaOrigen         =
DriveApp.getFolderById(ID_CARPETA_INVENTARIOS);
        const  archivos  =  carpetaOrigen.searchFiles(\`title  contains
'\${NOMBRE_ARCHIVO_INVENTARIO}'\`);

     if  (!archivos.hasNext())  return "Error:  No  se  encontró  el  archivo
de inventario.";

    const archivoExcel = archivos.next();
    const spreadsheet = SpreadsheetApp.openById(archivoExcel.getId());
    const sheet = spreadsheet.getSheets()[0];
    const data = sheet.getDataRange().getValues();

                      const     headers     =     data[0].map(h     =>
h.toString().toLowerCase().trim().replace(/\\s+/g, "_"));
                                    const         carpetaDestino         =
DriveApp.getFolderById(ID_CARPETA_JSON_DESTINO);

    let contador = 0;

    for (let i = 1; i < data.length; i++) {
      let fila = data[i];
      let objetoProceso = {};

      headers.forEach((header, index) => {
        if(fila[index]) objetoProceso[header] = fila[index];
      });

       if (objetoProceso.codigo_nivel_1 || objetoProceso.codigo_nivel_2
|| objetoProceso.codigo_nivel_3) {
                  let  codigoFinal  =  objetoProceso.codigo_nivel_3  ||
objetoProceso.codigo_nivel_2 || objetoProceso.codigo_nivel_1;
```

```
        let nombreArchivo = \`\${codigoFinal}.json\`;

                                carpetaDestino.createFile(nombreArchivo,
JSON.stringify(objetoProceso, null, 2), MimeType.PLAIN_TEXT);
        contador++;
      }
    }
       return \`¡Éxito! Se han generado \${contador} archivos JSON
correctamente.\`;
  } catch (error) {
    return \`Error crítico: \${error.message}\`;
  }
}`;

  return (
    <div className="flex h-screen bg-gray-50 font-sans text-gray-800">
      {/* Sidebar */}
         <div className="w-64 bg-white border-r border-gray-200 flex
flex-col shadow-sm">
        <div className="p-6 border-b border-gray-200">
               <h1 className="text-xl font-bold text-emerald-700 flex
items-center gap-2">
            <Database className="w-6 h-6" />
            SGP - Módulo
          </h1>
            <p className="text-xs text-gray-500 mt-1">Automatización de
Fichas</p>
        </div>

        <nav className="flex-1 p-4 space-y-2">
          <button
            onClick={() => setActiveTab('dashboard')}
                 className={`w-full flex items-center gap-3 px-4 py-3
rounded-lg text-sm font-medium transition-colors ${activeTab ===
'dashboard' ? 'bg-emerald-50 text-emerald-700' : 'text-gray-600
hover:bg-gray-100'}`}
          >
            <Play className="w-5 h-5" />
            Simulador
          </button>
          <button
            onClick={() => setActiveTab('codigo')}
```

```
                  className={`w-full flex items-center gap-3 px-4 py-3
rounded-lg text-sm font-medium transition-colors ${activeTab ===
'codigo' ? 'bg-emerald-50 text-emerald-700' : 'text-gray-600
hover:bg-gray-100'}`}
          >
            <Code className="w-5 h-5" />
            Código para Apps Script
          </button>
        </nav>

        <div className="p-4 border-t border-gray-200">
                    <div className="flex items-center gap-2 text-xs
text-gray-400">
            <CheckCircle className="w-4 h-4 text-emerald-500" />
            Alineado a la NT N° 002-2025-PCM/SGP
          </div>
        </div>
      </div>

      {/* Main Content */}
      <div className="flex-1 overflow-hidden flex flex-col">

        {/* Header */}
                  <header className="bg-white p-6 shadow-sm border-b
border-gray-200 flex justify-between items-center z-10">
          <h2 className="text-2xl font-semibold text-gray-800">
                  {activeTab === 'dashboard' ? 'Panel de Ejecución' :
'Implementación Técnica'}
          </h2>
                    <div className="flex items-center gap-4 text-sm
text-gray-500">
            <span>Usuario: Especialista Modernización</span>
                  <div className="w-8 h-8 rounded-full bg-emerald-100
text-emerald-700 flex items-center justify-center font-bold">EM</div>
          </div>
        </header>

        {/* Content Area */}
        <main className="flex-1 overflow-y-auto p-8">

          {activeTab === 'dashboard' && (
            <div className="max-w-5xl mx-auto space-y-6">
```

```
              {/* Card de Acción Principal */}
                <div className="bg-white p-8 rounded-xl shadow-sm border
border-gray-200 flex flex-col items-center justify-center text-center">
                  <div className="w-16 h-16 bg-blue-50 rounded-full flex
items-center justify-center mb-4">
                    <FolderDown className="w-8 h-8 text-blue-600" />
                  </div>
                    <h3 className="text-xl font-bold mb-2">Carga Dinámica
de Inventario (JSON)</h3>
                  <p className="text-gray-500 max-w-lg mb-8">
                      Carga un archivo JSON estructurado con tu inventario
de procesos. La aplicación leerá dinámicamente las llaves (columnas) y
armará las vistas sin depender de constantes.
                  </p>

                  <input
                    type="file"
                    id="json-upload"
                    accept=".json"
                    className="hidden"
                    onChange={handleFileUpload}
                  />
                  <label
                    htmlFor="json-upload"
                        className={`flex items-center justify-center gap-2
px-8 py-4 rounded-lg font-bold text-white transition-all shadow-md
cursor-pointer ${
                      isProcessing
                      ? 'bg-gray-400 cursor-not-allowed'
                                  : 'bg-emerald-600 hover:bg-emerald-700
hover:shadow-lg hover:-translate-y-0.5'
                    }`}
                  >
                    {isProcessing ? (
                      <>
                            <div className="w-5 h-5 border-2 border-white
border-t-transparent rounded-full animate-spin"></div>
                        Leyendo archivo JSON...
                      </>
                    ) : (
                      <>
                        <FileJson className="w-5 h-5" />
```

```
                        {fileName ? `Cargado: ${fileName}` : 'Seleccionar
Archivo JSON'}
                      </>
                    )}
                  </label>
                </div>

                {/* Resultados */}
                {procesosGenerados.length > 0 && (
                    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6
animate-in fade-in slide-in-from-bottom-4 duration-500">

                    {/* Tabla de Procesos */}
                        <div className="lg:col-span-2 bg-white rounded-xl
shadow-sm border border-gray-200 overflow-hidden">
                                <div className="p-4 bg-gray-50 border-b
border-gray-200 flex justify-between items-center">
                            <h4 className="font-semibold text-gray-700 flex
items-center gap-2">
                                        <CheckCircle className="w-5 h-5
text-emerald-500" />
                                            Archivos Generados en Drive
({procesosGenerados.length})
                        </h4>
                      </div>
                      <div className="overflow-x-auto">
                              <table className="w-full text-sm text-left
text-gray-500">
                                <thead className="text-xs text-gray-700
uppercase bg-gray-50">
                            <tr>
                                        {procesosGenerados.length > 0 &&
Object.keys(procesosGenerados[0]).slice(0, 4).map((key, index) => (
                                         <th key={index} className="px-6
py-3">{key.replace(/_/g, ' ')}</th>
                                ))}
                                                <th className="px-6 py-3
text-right">Acción</th>
                            </tr>
                          </thead>
                          <tbody>
                            {procesosGenerados.map((proceso, index) => {
```

```
                                // Extraemos solo las primeras 4 llaves
dinámicamente para no desbordar la tabla
                              const keys = Object.keys(proceso).slice(0,
4);
                            return (
                                    <tr key={index} className="bg-white
border-b hover:bg-emerald-50 transition-colors">
                                {keys.map((key, cellIndex) => (
                                    <td key={cellIndex} className={`px-6
py-4  truncate  max-w-[200px]  ${cellIndex  ===  0  ?  'font-medium
text-gray-900 whitespace-nowrap' : ''}`} title={proceso[key]}>
                                    {String(proceso[key])}
                                  </td>
                                ))}
                                <td className="px-6 py-4 text-right">
                                  <button
                                                          onClick={()  =>
setJsonSeleccionado(proceso)}
                                                  className="text-emerald-600
hover:text-emerald-900 font-medium inline-flex items-center gap-1"
                                  >
                                    Ver Estructura
                                  </button>
                                </td>
                              </tr>
                            )
                          })}
                        </tbody>
                      </table>
                    </div>
                  </div>

                  {/* Vista Previa del JSON */}
                      <div className="bg-slate-900 rounded-xl shadow-sm
border border-slate-800 overflow-hidden flex flex-col">
                     <div className="p-4 border-b border-slate-700 flex
justify-between items-center">
                      <h4 className="font-mono text-sm text-emerald-400
flex items-center gap-2">
                        <FileJson className="w-4 h-4" />
                        Vista Previa JSON
                      </h4>
                    </div>
```

```
                    <div className="p-4 flex-1 overflow-y-auto">
                      {jsonSeleccionado ? (
                                <pre className="text-xs text-slate-300
font-mono whitespace-pre-wrap break-words">
                          {JSON.stringify(jsonSeleccionado, null, 2)}
                        </pre>
                      ) : (
                                  <div className="h-full flex flex-col
items-center justify-center text-slate-500 text-sm">
                                   <AlertCircle className="w-8 h-8 mb-2
opacity-50" />
                           Selecciona un proceso de la tabla para ver su
estructura JSON.
                        </div>
                      )}
                    </div>
                  </div>

                </div>
              )}
            </div>
          )}

          {activeTab === 'codigo' && (
            <div className="max-w-4xl mx-auto space-y-8">
                  <div className="bg-blue-50 border border-blue-200 p-4
rounded-lg text-blue-800 flex items-start gap-3">
                   <AlertCircle className="w-6 h-6 flex-shrink-0 mt-0.5"
/>
                <div>
                             <h4 className="font-bold">Instrucciones de
Implementación</h4>
                      <p className="text-sm mt-1">Copia estos bloques de
código en tu proyecto de Google Apps Script (script.google.com). No
olvides cambiar los IDs de las carpetas por los tuyos.</p>
                </div>
              </div>

              {/* Bloque 1 */}
                      <div className="bg-slate-900 rounded-xl shadow-sm
overflow-hidden">
                   <div className="flex justify-between items-center p-3
border-b border-slate-700 bg-slate-800">
```

```
                            <span  className="text-slate-300  font-mono
text-sm">Código.gs  (Backend)</span>
                  <button
                                onClick={()  =>  copiarCodigo('backend',
codigoBackend)}
                        className="text-slate-400 hover:text-white flex
items-center   gap-1   text-xs   px-2   py-1   bg-slate-700   rounded
transition-colors"
                  >
                                {copiado  ===  'backend'  ?  <CheckCircle
className="w-4 h-4 text-emerald-400" />  :  <Copy className="w-4 h-4" />}
                    {copiado === 'backend' ? 'Copiado' : 'Copiar'}
                  </button>
                </div>
                <div className="p-4 overflow-x-auto">
                  <pre className="text-sm text-emerald-400 font-mono">
                    <code>{codigoBackend}</code>
                  </pre>
                </div>
              </div>
            </div>
          )}

        </main>
      </div>
    </div>
  );
};

export default App;
```

Consulta. Debes de dejar la estructura debe de ser dinámica y no ser una constante para que partir de un archivo JSON que el aplicativo debe de leer

¡Excelente decisión arquitectónica! Pasar de constantes fijas a un modelo donde la estructura es inferida dinámicamente desde el archivo de origen (JSON) es la esencia de una aplicación escalable.

Para lograr esto, he modificado el aplicativo. Hemos eliminado las constantes "duras" y agregado una lógica de **lectura dinámica de archivos JSON**.

Ahora:

1. El aplicativo cuenta con un botón para **cargar un archivo** `.json` desde tu computadora.
2. Lee dinámicamente las llaves (propiedades) del JSON subido.

3. Construye la tabla de visualización basándose estrictamente en las propiedades que encuentre, sin importar cómo se llamen las columnas de tu inventario.

Aquí tienes los cambios (diff) aplicados al archivo principal para implementar esta flexibilidad:

¿Cómo probarlo?

1. Crea un archivo en tu computadora llamado prueba.json.
2. Pega dentro un arreglo JSON con las columnas que tú quieras (por ejemplo: [{"mi_codigo": "A.1", "nombre": "Prueba"}, {"mi_codigo": "B.1", "nombre": "Prueba 2"}]).
3. Ve a la vista previa del aplicativo a la derecha, haz clic en **"Seleccionar Archivo JSON"** y sube tu archivo.
4. Verás cómo la tabla y el visor construyen la vista interpretando las nuevas columnas sin que se haya programado nada de forma fija.

Consulta. Pasame el modelo del formato JSON para las fichas de Productos y Procesos y el formato JSON para el inventario de productos y procesos, la primera tarea será crear carpetas y los documentos con Google Docs y las segundas sería ir registrando los documentos a medida de los ingreso al aplicativo

Para organizar tu trabajo de acuerdo a la **Norma Técnica N° 002-2025-PCM/SGP** para la Gestión por Procesos, vamos a estructurar los dos modelos de datos que necesitas.

Es importante entender la diferencia:

1. **El Inventario de Procesos:** Es un *arreglo (array)* que contiene la lista o resumen de todos tus procesos en una sola vista (como una sábana de Excel).
2. **La Ficha de Proceso/Producto:** Es un *objeto individual (object)* que describe el detalle íntimo (entradas, salidas, recursos, flujograma) de un único proceso de manera profunda.

A continuación, te presento ambos modelos JSON alineados a las exigencias de la Presidencia del Consejo de Ministros (PCM) peruana.

1. Modelo JSON para el Inventario General (Array de Procesos)

Este formato representa tu tabla general. Es ideal para que la aplicación web lo lea, construya tu "Mapa de Procesos" y liste todos los documentos que tienes pendientes de detallar.

JSON

```
[
  {
    "id_unico": "P-M01-01",
    "codigo_nivel_0": "M.01",
    "nombre_nivel_0": "Gestión de la Promoción Agraria",
    "codigo_nivel_1": "M.01.01",
```

```
    "nombre_nivel_1": "Capacitación para el fortalecimiento de capacidades asociativas",
    "codigo_nivel_2": "",
    "nombre_nivel_2": "",
    "tipo_proceso": "Misional",
    "dueno_proceso": "Unidad de Promoción y Desarrollo de Capacidades",
    "producto_principal": "Capacitación ejecutada para productores agrarios",
    "estado_documentacion": "Pendiente",
    "version_vigente": "1.0",
    "fecha_ultima_actualizacion": "18/04/2026"
  },
  {
    "id_unico": "P-S01-01",
    "codigo_nivel_0": "S.01",
    "nombre_nivel_0": "Gestión Administrativa",
    "codigo_nivel_1": "S.01.01",
    "nombre_nivel_1": "Gestión de Recursos Humanos",
    "codigo_nivel_2": "S.01.01.02",
    "nombre_nivel_2": "Selección y Contratación de Personal",
    "tipo_proceso": "Soporte",
    "dueno_proceso": "Oficina de Administración",
    "producto_principal": "Personal idóneo contratado",
    "estado_documentacion": "Aprobado",
    "version_vigente": "2.1",
    "fecha_ultima_actualizacion": "10/01/2026"
  }
]
```


2. Modelo JSON para la Ficha Técnica de Proceso y Producto

Este modelo es para un archivo individual (ej. `ficha_M.01.01.json`). Aquí es donde desglosas la "Caracterización del Proceso" tal como lo exige el Anexo de la Norma Técnica, definiendo proveedores, entradas, salidas y clientes (modelo SIPOC extendido).

JSON


```
{
  "metadatos_documento": {
    "codigo_documento": "PE-M.01.01-F01",
    "version": "1.0",
    "fecha_aprobacion": "18/04/2026",
    "estado": "En revisión"
  },
  "datos_generales": {
    "codigo_proceso": "M.01.01",
    "denominacion_proceso": "Capacitación para el fortalecimiento de capacidades asociativas",
    "tipo_proceso": "Misional",
    "dueno_proceso": "Jefe de la Unidad de Promoción y Desarrollo de Capacidades",
     "objetivo": "Desarrollar y fortalecer las competencias operativas, comerciales y organizativas de los productores agrarios beneficiarios de AGROIDEAS.",
```

"alcance_inicio": "Recepción de la solicitud de capacitación por parte de la organización agraria.",
"alcance_fin": "Emisión del informe de evaluación de resultados de la capacitación ejecutada.",
"base_legal": [
"Ley N° 29337, Ley de Competitividad Productiva Agraria.",
"Manual de Operaciones de AGROIDEAS vigente."
]
},
"caracterizacion_elementos": {
"proveedores": [
{
"tipo": "Externo",
"nombre": "Organizaciones Agrarias (OA)",
"insumo_entregado": "Solicitud de requerimiento de capacitación y listado de participantes."
},
{
"tipo": "Interno",
"nombre": "Unidad de Planificación y Presupuesto",
"insumo_entregado": "Presupuesto anual aprobado (POI) para contrataciones."
}
],
"entradas": [
"Términos de Referencia (TDR) elaborados",
"Padrón de beneficiarios validado",
"Material didáctico propuesto"
],
"salidas_productos": [
{
"nombre_producto_oficial": "Capacitación para fortalecimiento de capacidades empresariales productivas ejecutada",
"unidad_medida": "Número de capacitaciones",
"frecuencia": "Mensual",
"receptor_cliente": "Productores Agrarios / Organizaciones Agrarias (OA)"
},
{
"nombre_producto_oficial": "Informe de Evaluación de Satisfacción y Aprendizaje",
"unidad_medida": "Documento",
"frecuencia": "Por cada evento",
"receptor_cliente": "Dirección Ejecutiva de AGROIDEAS"
}
]
},
"controles_e_indicadores": [
{
"nombre_indicador": "Porcentaje de cumplimiento del Plan Anual de Capacitaciones",
"formula": "(Número de capacitaciones ejecutadas / Número de capacitaciones programadas) * 100",
"meta": "90%",
"frecuencia_medicion": "Trimestral"

```
    }
  ],
  "recursos_necesarios": {
    "sistemas_informaticos": ["Sistema ESV-SIPA", "Módulo de Trámite Documentario"],
    "infraestructura": ["Aulas de capacitación locales", "Equipos multimedia portátiles"]
  },
  "riesgos_asociados": [
    "Baja asistencia de los productores a los eventos programados debido a factores climáticos."
  ]
}
```


¿Cómo organizar tus carpetas en Google Drive?

Para mantener el orden a medida que vayas registrando estos documentos, te sugiero esta estructura de directorios en tu Drive:

- 📂 **SGP_Gestion_por_Procesos_AGROIDEAS**
  - 📄 00_Inventario_Maestro_Procesos.xlsx (o .json)
  - 📂 **01_Plantillas_Oficiales**
    - 📄 PL-01_Ficha_Proceso_V1.docx (o Google Doc)
    - 📄 PL-02_Ficha_Procedimiento_V1.docx
  - 📂 **02_Fichas_Tecnicas_JSON** (Aquí tu aplicativo guardará los datos crudos)
    - 📂 Misionales
      - 📄 M.01.01_capacitacion.json
    - 📂 Soporte
    - 📂 Estrategicos
  - 📂 **03_Documentos_Generados_PDF** (Las fichas ya armadas para firmar)

Este modelo de datos te permite cumplir con la fase 3 ("Caracterización de los procesos") y la elaboración de la "Ficha técnica de proceso y producto" que menciona el manual de implementación de la PCM.

Consulta. Esta es el archivo del inventario de procesos y procedimientos https://docs.google.com/spreadsheets/d/1DqkqD9I67dK-taTUAcPk7K6iWOdXqbRy/edit?usp=drivesdk&ouid=101701601019729543859&rtpof=true&sd=true para que la evalúes y formules la mejor forma de procesarlo y generar el archivo JSON correspondiente

He evaluado el contenido de tu archivo **"Inventario de Procesos y Procedimientos Nivel 1 a + 28_03.xlsx"** (y sus versiones relacionadas en tu Drive).

Evaluación del Archivo

Tras analizar la estructura del documento, he detectado lo siguiente:

1. **Columnas Identificadas:** El archivo cuenta con una estructura muy clara de 9 a 10 columnas que mapean desde el Nivel 0 (Macroproceso) hasta el Nivel 3 (Procedimiento), incluyendo los productos oficiales.
   - Código Nivel 0

    - Denominación Nivel 0 (Ficha Oficial)
    - Código Nivel 1
    - Denominación Nivel 1 (Ficha Oficial)
    - Productos Oficiales Asociados (RDE-000050)
    - Código Nivel 2
    - Denominación Nivel 2 (Ficha Oficial)
    - Código Nivel 3
    - Denominación Nivel 3 (Propuesto)
    - Notas (en algunas versiones del documento).
2. **Distribución en Pestañas:** La información no está en una sola "sábana" gigante, sino que está segmentada en diferentes hojas o pestañas según el macroproceso (por ejemplo, hojas nombradas como "E.01", "Inv-E.01").
3. **Inicio de los Datos:** Los encabezados principales parecen estar entre la fila 2 y 3, y los datos reales comienzan a partir de la fila 4.

La Mejor Forma de Procesarlo

Dado que tienes múltiples pestañas, la mejor estrategia es crear un script que **itere sobre todas las hojas del documento**, consolide las filas que tengan datos válidos (ignorando filas vacías) y genere un JSON maestro, o varios JSONs individuales.

Para asegurar que las llaves del JSON sean fáciles de leer por tu aplicativo web (sin espacios ni caracteres especiales), el procesamiento debe "limpiar" los títulos de las columnas.

1. El Formato JSON Resultante (Estructura Propuesta)

Basándome estrictamente en los encabezados reales de tu archivo, este sería el formato JSON perfecto para cada fila (proceso/procedimiento) de tu inventario:

```JSON
{
  "codigo_nivel_0": "E.01",
  "denominacion_nivel_0_ficha_oficial": "Gestión Estratégica",
  "codigo_nivel_1": "E.01.01",
  "denominacion_nivel_1_ficha_oficial": "Gestión del planeamiento institucional",
  "productos_oficiales_asociados_rde_000050": "Programación operativa institucional multianual",
  "codigo_nivel_2": "E.01.01.01",
  "denominacion_nivel_2_ficha_oficial": "Diseño de la política y programación de planes institucionales",
  "codigo_nivel_3": "",
  "denominacion_nivel_3_propuesto": "",
  "notas": ""
}
```

2. Actualización del Script de Procesamiento

Para que el script de Google Apps Script que te di anteriormente funcione **perfectamente** con la estructura real de tu archivo (leyendo todas las pestañas), debemos ajustarlo de esta manera:

JavaScript

```
function procesarInventarioCompleto() {
  const ID_ARCHIVO = '1DqkqD9I67dK-taTUAcPk7K6iWOdXqbRy'; // El ID de tu archivo
   const ID_CARPETA_DESTINO = 'PONER_AQUI_ID_CARPETA_DESTINO'; // Carpeta donde
se guardarán los JSON

  const spreadsheet = SpreadsheetApp.openById(ID_ARCHIVO);
  const hojas = spreadsheet.getSheets(); // Obtenemos todas las pestañas
  const carpetaDestino = DriveApp.getFolderById(ID_CARPETA_DESTINO);

  let totalProcesados = 0;
  let inventarioMaestro = []; // Array para guardar todo en un solo JSON si lo deseas

  // Recorrer cada pestaña del Excel
  hojas.forEach(hoja => {
    // Evitar procesar hojas ocultas o vacías
    if (hoja.isSheetHidden() || hoja.getLastRow() < 4) return;

      // Obtenemos todo el rango de datos. Asumimos que los encabezados están en la fila 3
(índice 2)
    const data = hoja.getDataRange().getValues();
    const headersRaw = data[2]; // Fila de los encabezados (ajustar a 1 si están en la fila 2)

    // Función para normalizar las columnas (ej: "Código Nivel 0" -> "codigo_nivel_0")
    const headers = headersRaw.map(h => {
      return h.toString().toLowerCase().trim()
            .replace(/[\n\r]/g, '_') // Quita saltos de línea
            .replace(/[\(\)-]/g, '') // Quita paréntesis y guiones
            .replace(/\s+/g, '_');   // Cambia espacios por guiones bajos
    });

    // Empezamos a leer desde la fila 4 (índice 3)
    for (let i = 3; i < data.length; i++) {
      let fila = data[i];
      let objetoProceso = {};
      let tieneDatos = false;

      headers.forEach((header, index) => {
        if(header !== "" && fila[index] !== undefined) {
          objetoProceso[header] = fila[index];
          if(fila[index] !== "") tieneDatos = true;
        }
      });

      // Solo guardar si la fila no está en blanco
```

```
    if (tieneDatos && objetoProceso.codigo_nivel_0) {
      inventarioMaestro.push(objetoProceso);

      // Determinar el código más profundo para nombrar el archivo
            let codigoFinal = objetoProceso.codigo_nivel_3 || objetoProceso.codigo_nivel_2 || objetoProceso.codigo_nivel_1 || objetoProceso.codigo_nivel_0;
      let nombreArchivo = `Ficha_${codigoFinal}.json`;

      // Crear el archivo JSON individual en la carpeta
              carpetaDestino.createFile(nombreArchivo, JSON.stringify(objetoProceso, null, 2), MimeType.PLAIN_TEXT);
      totalProcesados++;
    }
  }
});

// (Opcional) Crear un único archivo JSON maestro con todo el inventario
                                carpetaDestino.createFile('Inventario_Maestro_Completo.json', JSON.stringify(inventarioMaestro, null, 2), MimeType.PLAIN_TEXT);

 console.log(`Proceso terminado. Se generaron ${totalProcesados} archivos JSON individuales y 1 JSON Maestro.`);
}
```

Ventaja de esta estructura

Con este método lograrás dos cosas simultáneamente para tu aplicativo:

1. **JSON Maestro:** Tendrás un archivo Inventario_Maestro_Completo.json que contiene un arreglo con **todos** los procesos. Este es el archivo ideal que tu aplicativo web debe leer de una sola vez para construir la tabla que te mostré en el prototipo anterior.
2. **JSONs Individuales:** Tendrás archivos como Ficha_E.01.01.01.json. Estos archivos específicos son los que usarás posteriormente cuando quieras combinar la data con la plantilla de Google Docs para generar los documentos formales de la Ficha de Proceso listos para imprimir y firmar.